Sesión 23: Serialización de Datos (Modelado de Datos JSON, Análisis Estático de Tipos, Uso de `json_serializable` y `freezed`)

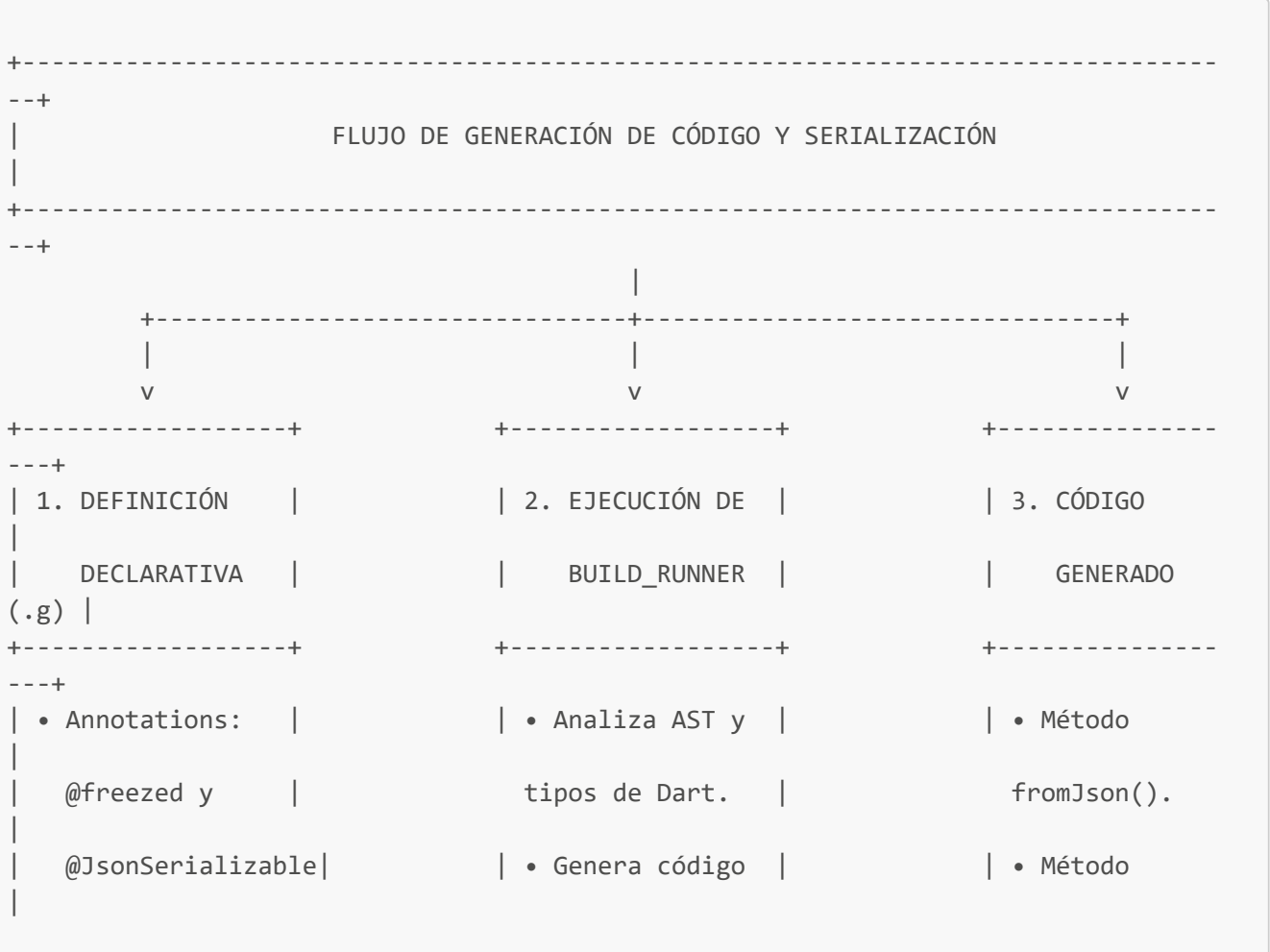
Introducción

En el desarrollo de aplicaciones robustas en Flutter, el manejo de datos provenientes de APIs REST o almacenamiento local requiere transformar estructuras no tipadas (`Map<String, dynamic>`) en objetos fuertemente tipados. Escribir manualmente métodos de serialización (`fromJson` / `toJson`), implementaciones de inmutabilidad (`copyWith`) y sobreescrituras de igualdad (`operator ==` y `hashCode`) resulta propenso a errores en tiempo de ejecución y dificulta el mantenimiento.

En esta sesión abordaremos la **Serialización de Datos** utilizando metaprogramación y generación de código en Dart. Analizaremos cómo aprovechar el análisis estático de tipos para garantizar la seguridad en tiempo de compilación y utilizaremos las herramientas `json_serializable` y `freezed` junto con `build_runner` para automatizar la creación de modelos de datos inmutables y seguros.

Conceptos y Definiciones

El ecosistema de Dart permite pasar de mapeos manuales y frágiles a modelos declarativos que generan su infraestructura mediante el análisis de tipos.



```
| • Clases y       |              boilerplate en   |              toJson().
|
|   Constructores  |              ficheros part    |            | • copyWith(),
|
|   Factory.       |              '*.freezed.dart' |              equals, y
|
|                  |              y '*.g.dart'.    |              toString().
|
+------------------+            +------------------+            +--------------
---+
```

1. Análisis Estático de Tipos y Seguridad Tipográfica

- **Serialización Manual vs. Generada:** El parseo manual depende de cadenas de texto explícitas para acceder a llaves de un mapa, lo que expone la aplicación a excepciones `TypeError` o `NullThrownError` en tiempo de ejecución. La generación de código valida la estructura durante la compilación.
- **Archivos Parte (`part` y `part of`):** Mecanismo de Dart que permite dividir una biblioteca en múltiples archivos. Las herramientas de generación crean archivos complementarios (ej. `alumno.freezed.dart` y `alumno.g.dart`) que se integran de forma transparente a la clase principal.

---

2. Paquete `json_serializable`

- **Propósito:** Genera automáticamente el código necesario para convertir objetos de Dart hacia y desde formatos JSON.
- **Anotaciones Clave:**
- `@JsonSerializable()`: Indica al generador que debe procesar la clase.
- `@JsonKey(name: 'key_name')`: Mapea campos de Dart (`camelCase`) con claves JSON externas (`snake_case` o con caracteres especiales).
- `@JsonKey(includeToJson: false)`: Omite atributos específicos al serializar hacia JSON.

---

3. Paquete `freezed`

- **Propósito:** Generador de código para modelos de datos inmutables, uniones de datos (Unions/Sealed classes) y superpoderes de evaluación de igualdad y copia.
- **Características Principales:**
- **Inmutabilidad Absoluta:** Los objetos no pueden ser modificados una vez instanciados.
- **Método `copyWith`:** Permite clonar un objeto modificando solo campos específicos.
- **Deep Equality:** Sobrescribe `operator ==` y `hashCode` para comparar valores en lugar de referencias de memoria.
- **Pattern Matching:** Ofrece métodos como `when`, `maybeWhen`, `map` y `maybeMap` para manejar estados y uniones complejas.

Ejemplo Completo con Código Fuente: Modelo de Alumno ESCOM con `freezed` y `json_serializable`

El siguiente ejemplo demuestra la definición completa de un modelo de datos inmutable utilizando `freezed` e integración con `json_serializable`. Se incluye la estructura que simula el código generado para poder ejecutar la aplicación demostrativa de manera autónoma en un único archivo.

> **Configuración del proyecto (`pubspec.yaml`):**
>
> ```
> dependencies:
>   flutter:
>     sdk: flutter
>   freezed_annotation: ^2.4.1
>   json_annotation: ^4.9.0
>
> dev_dependencies:
>   build_runner: ^2.4.9
>   freezed: ^2.5.2
>   json_serializable: ^6.8.0
> ```
>
> *Comando para generación de código:* `dart run build_runner build --delete-conflicting-outputs`

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoSerializacionApp());
}

// ============================================================================
// DEFINICIÓN DECLARATIVA DEL MODELO CON FREEZED Y JSON_SERIALIZABLE
// ============================================================================
// En un entorno de producción con generación de código, la clase se declara así:
//
// import 'package:freezed_annotation/freezed_annotation.dart';
// part 'alumno.freezed.dart';
// part 'alumno.g.dart';
//
// @freezed
// class Alumno with _$Alumno {
//   const factory Alumno({
//     required String boleta,
//     @JsonKey(name: 'nombre_completo') required String nombreCompleto,
//     required String carrera,
//     @Default(0.0) double promedio,
//   }) = _Alumno;
//
//   factory Alumno.fromJson(Map<String, dynamic> json) => _$AlumnoFromJson(json);
// }
```

```dart
/// Simulación estructural del comportamiento inmutable y seguro generado por Freezed/JsonSerializable
@immutable
class Alumno {
  final String boleta;
  final String nombreCompleto;
  final String carrera;
  final double promedio;

  const Alumno({
    required this.boleta,
    required this.nombreCompleto,
    required this.carrera,
    this.promedio = 0.0,
  });

  /// Mapeo equivalente al generado por json_serializable (@JsonKey)
  factory Alumno.fromJson(Map<String, dynamic> json) {
    return Alumno(
      boleta: json['boleta'] as String? ?? '',
      nombreCompleto: json['nombre_completo'] as String? ?? 'Sin Nombre',
      carrera: json['carrera'] as String? ?? 'Ingeniería en Sistemas Computacionales',
      promedio: (json['promedio'] as num?)?.toDouble() ?? 0.0,
    );
  }

  /// Serialización equivalente a la generada por json_serializable
  Map<String, dynamic> toJson() {
    return {
      'boleta': boleta,
      'nombre_completo': nombreCompleto,
      'carrera': carrera,
      'promedio': promedio,
    };
  }

  /// Método copyWith equivalente al generado automáticamente por Freezed
  Alumno copyWith({
    String? boleta,
    String? nombreCompleto,
    String? carrera,
    double? promedio,
  }) {
    return Alumno(
      boleta: boleta ?? this.boleta,
      nombreCompleto: nombreCompleto ?? this.nombreCompleto,
      carrera: carrera ?? this.carrera,
      promedio: promedio ?? this.promedio,
    );
  }

  /// Comparación estructural por valores (Value Equality de Freezed)
  @override
```

```
  bool operator ==(Object other) =>
      identical(this, other) ||
      other is Alumno &&
          runtimeType == other.runtimeType &&
          boleta == other.boleta &&
          nombreCompleto == other.nombreCompleto &&
          carrera == other.carrera &&
          promedio == other.promedio;

  @override
  int get hashCode =>
      boleta.hashCode ^
      nombreCompleto.hashCode ^
      carrera.hashCode ^
      promedio.hashCode;

  @override
  String toString() {
    return 'Alumno(boleta: $boleta, nombreCompleto: $nombreCompleto, carrera:
$carrera, promedio: $promedio)';
  }
}

// ============================================================================
// INTERFAZ DE USUARIO Y DEMOSTRACIÓN DE FUNCIONALIDADES
// ============================================================================

class DemoSerializacionApp extends StatelessWidget {
  const DemoSerializacionApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Freezed & JsonSerializable',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaSerializacion(),
    );
  }
}

class PantallaSerializacion extends StatefulWidget {
  const PantallaSerializacion({super.key});

  @override
  State<PantallaSerializacion> createState() => _PantallaSerializacionState();
}

class _PantallaSerializacionState extends State<PantallaSerializacion> {
```

```
  Alumno? _alumnoActual;
  String _jsonStringDisplay = '';
  String _logComparacion = '';

  // JSON crudo que simula la respuesta de la API REST
  final Map<String, dynamic> _rawJsonApi = {
    'boleta': '2026630001',
    'nombre_completo': 'Margaret Hamilton',
    'carrera': 'Ingeniería en Sistemas Computacionales',
    'promedio': 9.85,
  };

  void _deserializarJson() {
    setState(() {
      // Conversión segura de JSON a Objeto fuertemente tipado
      _alumnoActual = Alumno.fromJson(_rawJsonApi);
      _jsonStringDisplay = _rawJsonApi.toString();
      _logComparacion = 'Objeto Alumno creado exitosamente desde JSON.';
    });
  }

  void _modificarConCopyWith() {
    if (_alumnoActual == null) return;

    setState(() {
      // Uso del patrón copyWith para crear una nueva instancia modificada
      final alumnoModificado = _alumnoActual!.copyWith(
        promedio: 10.0,
        nombreCompleto: 'Margaret Hamilton (Honorífica)',
      );

      _logComparacion = 'Instancia original: ${_alumnoActual!.promedio}\n'
          'Nueva instancia (copyWith): ${alumnoModificado.promedio}\n'
          '¿Son idénticos en memoria?: ${identical(_alumnoActual,
alumnoModificado)}\n'
          '¿Son iguales por contenido?: ${_alumnoActual == alumnoModificado}';

      _alumnoActual = alumnoModificado;
      _jsonStringDisplay = _alumnoActual!.toJson().toString();
    });
  }

  void _probarIgualdadEstructural() {
    final alumno1 = Alumno.fromJson(_rawJsonApi);
    final alumno2 = Alumno.fromJson(_rawJsonApi);

    setState(() {
      _logComparacion = 'Alumno 1: ${alumno1.boleta}\n'
          'Alumno 2: ${alumno2.boleta}\n'
          '¿Misma referencia de memoria?: ${identical(alumno1, alumno2)}\n'
          '¿Igualdad por contenido (==)?: ${alumno1 == alumno2}';
    });
  }
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Serialización & Inmutabilidad'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(20.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            ElevatedButton.icon(
              onPressed: _deserializarJson,
              icon: const Icon(Icons.download),
              label: const Text('1. Deserializar desde JSON (fromJson)'),
            ),
            const SizedBox(height: 10),

            ElevatedButton.icon(
              onPressed: _alumnoActual != null ? _modificarConCopyWith : null,
              icon: const Icon(Icons.edit),
              label: const Text('2. Actualizar Promedio con copyWith()'),
            ),
            const SizedBox(height: 10),

            ElevatedButton.icon(
              onPressed: _probarIgualdadEstructural,
              icon: const Icon(Icons.compare_arrows),
              label: const Text('3. Probar Value Equality (==)'),
            ),
            const SizedBox(height: 20),

            if (_alumnoActual != null) ...[
              Card(
                elevation: 3,
                child: Padding(
                  padding: const EdgeInsets.all(16.0),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        _alumnoActual!.nombreCompleto,
                        style: Theme.of(context).textTheme.titleMedium?.copyWith(
                              fontWeight: FontWeight.bold,
                            ),
                      ),
                      const SizedBox(height: 8),
                      Text('Boleta: ${_alumnoActual!.boleta}'),
                      Text('Carrera: ${_alumnoActual!.carrera}'),
                      Text('Promedio: ${_alumnoActual!.promedio}'),
                    ],
                  ),
```

```
                ),
              ),
              const SizedBox(height: 16),
            ],

            Text(
              'Representación JSON / Estado:',
              style: Theme.of(context).textTheme.titleSmall,
            ),
            Container(
              margin: const EdgeInsets.only(top: 8),
              padding: const EdgeInsets.all(12),
              decoration: BoxDecoration(
                color: Colors.grey.shade200,
                borderRadius: BorderRadius.circular(8),
              ),
              child: Text(
                _jsonStringDisplay.isEmpty ? 'Sin datos' : _jsonStringDisplay,
                style: const TextStyle(fontFamily: 'monospace', fontSize: 12),
              ),
            ),
            const SizedBox(height: 16),

            Text(
              'Log de Operaciones:',
              style: Theme.of(context).textTheme.titleSmall,
            ),
            Container(
              margin: const EdgeInsets.only(top: 8),
              padding: const EdgeInsets.all(12),
              decoration: BoxDecoration(
                color: const Color(0xFF1E1E1E),
                borderRadius: BorderRadius.circular(8),
              ),
              child: Text(
                _logComparacion.isEmpty ? 'Esperando acción...' : _logComparacion,
                style: const TextStyle(
                  color: Colors.greenAccent,
                  fontFamily: 'monospace',
                  fontSize: 12,
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Prácticas Guiadas / Laboratorio

Práctica 1: Configuración de Claves compuestas y Valores por Defecto con `@JsonKey`

1. Añade a la definición del modelo un atributo opcional `fechaIngreso` de tipo `DateTime?`.
2. Utiliza la anotación `@JsonKey(name: 'ingreso_at', defaultValue: null)` para mapear la propiedad desde un campo JSON con nomenclatura externa.
3. Define un convertidor personalizado mediante `JsonConverter<DateTime, String>` para transformar cadenas en formato ISO-8601 a objetos `DateTime` durante la fase de deserialización.

Práctica 2: Modelado de Estado de Red mediante Uniones Libres de `freezed`

1. Define una unión discriminada con `freezed` para representar el estado de una petición de red:

```
@freezed
class EstadoConsulta with _$EstadoConsulta {
  const factory EstadoConsulta.inicial() = _Inicial;
  const factory EstadoConsulta.cargando() = _Cargando;
  const factory EstadoConsulta.exito(Alumno alumno) = _Exito;
  const factory EstadoConsulta.error(String mensaje) = _Error;
}
```

2. Implementa en la interfaz de usuario el método `.when()` o `.maybeWhen()` para renderizar condicionalmente los widgets correspondientes a cada estado de forma exhaustiva y segura.

Resumen y Puntos Clave

- **Seguridad Tipográfica:** La generación de código elimina los errores de parseo manual en tiempo de ejecución al validar la estructura JSON contra el sistema de tipos de Dart.
- **Separación con `json_serializable`:** `@JsonKey` permite aislar el nombre de las propiedades de Dart (`camelCase`) de los formatos externos provistos por la API (`snake_case`).
- **Inmutabilidad con `freezed`:** Garantiza la inmutabilidad de los datos, previene efectos secundarios y proporciona métodos esenciales como `copyWith` y comparaciones estructurales por valor (`==`).
- **Automatización con `build_runner`:** Herramienta de línea de comandos que compila las anotaciones y genera los archivos complementarios (`.freezed.dart` y `.g.dart`).

Bibliografía y Referencias

1. **Remi Rousselet.** (2024). *Freezed: Code generation for immutable classes and data-classes*. Recuperado de: https://pub.dev/packages/freezed
2. **Google LLC.** (2024). *Serializing JSON manually and using code generation libraries*. Recuperado de: https://docs.flutter.dev/data-and-backend/json
3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.